NATIONAL ECONOMICS UNIVERSITY, HANOI
COLLEGE OF TECHNOLOGY
FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

CREDIT CARD FRAUD DETECTION

FINAL REPORT

Topic: Credit Card Fraud Detection

Handling Class Imbalance and Designing a Cost-Aware Operating Policy

Instructor: Ph.D. Nguyễn Trọng Nghĩa

Student:	Nguyễn Tuấn Sỹ	11230775
	Nguyễn Tiến Công	11230739
	Nguyễn Nho Thắng	11230778
	Nguyễn Quang Cường	11230740
	Nguyễn Gia Phong	11230768

Abstract. Fraud detection on transactional data is an imbalanced, cost-sensitive problem: only 0.167% of the 283,726 transactions in the ULB credit card dataset are fraudulent, so accuracy is uninformative and the operating point matters more than the classifier. We evaluate 27 configurations spanning five model families (Logistic Regression, Decision Tree, Random Forest, XGBoost, Isolation Forest) and five imbalance-handling techniques under a leakage-controlled workflow, and we translate the winning model into a deployable operating policy. Random Forest with SMOTE at a 1:100 ratio is selected (F1 = 0.8605, Average Precision = 0.8239), reaching 78.17% recall at a 95% precision floor, or 73.24% with a leakage-free threshold. Central to the business answer is the capacity constraint: at 141,863 transactions per day no single threshold can satisfy both the 95% precision floor and a 100-review-per-day analyst capacity — it either demands 1.9 analysts or throttles recall to 40.9%. A two-tier policy that auto-blocks above 0.810 and reviews only the 0.175–0.810 band resolves this, attaining 81.0% recall at USD 154.65 per 85,118 transactions with just 70 reviews per day — an operational gain over the cheapest single threshold near that operating point, which needs 232 reviews per day for comparable recall. We also quantify the effect of tuning the threshold on the test set, report an observation consistent with a wider hyper-parameter search overfitting the cross-validation folds, and demonstrate that permutation importance and SHAP both rank `Amount` first where the default impurity importance ranks it 21st of 35.

Keywords: credit card fraud detection; class imbalance; SMOTE; cost-sensitive learning; precision–recall trade-off; threshold calibration; operating policy; SHAP.

"All quantitative results reported in this document were produced by the authors' own pipeline. The source code, the six executed Jupyter notebooks and the underlying result tables are available on request; the random seed is fixed at 42 so that every figure in this report is exactly reproducible."

Hanoi, October 2026

Contents

1 Introduction **3**
- 1.1 The Problem and Its Urgency . . . 3
- 1.2 Research Motivation . . . 3
- 1.3 Research Questions . . . 3
- 1.4 Problem Definition . . . 3
- 1.5 Literature Review . . . 3
- 1.6 Contribution . . . 4
- 1.7 Contribution of Group Members . . . 4

2 Related Work and Background **4**
- 2.1 Class Imbalance and Resampling . . . 4
- 2.2 Model Families . . . 4
- 2.3 Metrics and Evaluation . . . 4
- 2.4 Cost Model . . . 4

3 Methodology **5**
- 3.1 Problem Formulation . . . 5
- 3.2 Pipeline Architecture . . . 5
- 3.3 Model Configurations . . . 6
- 3.4 Leakage Control . . . 6
- 3.5 Feature Set Ablation . . . 6

4 Results **7**
- 4.1 Data Quality and Class Imbalance . . . 7
- 4.2 Accuracy Is Not a Usable Metric . . . 7
- 4.3 Imbalance Benchmark: 27 Configurations . . . 7
- 4.4 Model Selection and the Cost of Over-Searching . . . 8
- 4.5 Threshold Selection and Leakage in Threshold Tuning . . . 8
- 4.6 Explainability . . . 9
- 4.7 Calibration of the Predicted Probability . . . 10

5 Cost Analysis and Business Decision **11**
- 5.1 Cost Model and Operating Constraints . . . 11
- 5.2 Comparison with Rule-Based Baselines . . . 11
- 5.3 Analyst Capacity: The Operational Constraint . . . 11
- 5.4 Score Everything: Amount-Based Filtering Is Counter-Productive . . . 12
- 5.5 A Two-Tier Operating Policy . . . 12
- 5.6 Scalability and Latency . . . 13
- 5.7 What Each Imbalance Technique Costs . . . 14
- 5.8 Sensitivity to the Review-Cost Assumption . . . 14
- 5.9 Answers to the Seven Business Questions . . . 15

6 Discussion **16**
- 6.1 Principal Findings . . . 16
- 6.2 Limitations . . . 16
- 6.3 Recommendations . . . 17

7 Conclusion **18**

A Reproducibility **19**

1 Introduction

1.1 The Problem and Its Urgency

Credit card fraud has become one of the fastest-growing financial crimes worldwide. The Nilson Report estimates global card fraud losses at **USD 27.33 billion in 2023**, with projections exceeding USD 40 billion by 2027 [14]. The problem is not only the direct loss. A 2024 LexisNexis study establishes that every **USD 1 lost to fraud costs financial institutions USD 4.45** to detect, prevent and compensate [11]. Beyond the direct financial damage, fraud has a corrosive effect on customer trust: a substantial share of fraud victims disengage from their bank within months of the incident.
Banks respond with two operational levers that both carry hidden costs. They *reject* suspicious transactions, which avoids fraud losses but punishes legitimate customers. They *manually review* borderline cases, which is accurate but consumes scarce human analyst time. Traditional rule-based systems — for example, "block every foreign transaction above $X" — are cheap to implement but are blunt instruments: they miss a large share of fraud while wrongly blocking ordinary customers. The core engineering problem is therefore not simply "detect fraud" but *decide, per transaction, how confident we are, and what action that confidence justifies.*

1.2 Research Motivation

This project applies a supervised learning pipeline to the canonical credit card fraud dataset [2], but the evaluation philosophy is deliberately business-facing rather than metric-facing. Three commitments follow from this:

1. **Accuracy is not a meaningful metric on this problem.** With a class ratio of roughly 1:599, a classifier that labels *every* transaction legitimate scores 99.83% accuracy while catching exactly zero frauds. We therefore evaluate Recall, Precision, F1 and Average Precision.
2. **Imbalance handling must be justified economically.** SMOTE [4], Random Undersampling, SMOTE-ENN [8] and class weighting are all routinely prescribed. We compare all of them and translate each into a monetary cost, because a technique that improves a metric but explodes the analyst workload is worthless to a bank.
3. **The output must be an operating decision, not a probability.** A bank needs to know which transactions to block automatically, which to send to an analyst, and which to approve — and what that policy costs per million transactions.

1.3 Research Questions

The project is organised around four questions, each of which is answered by a specific part of this report:

1. **RQ1 (Detection).** Which model family and imbalance-handling technique maximise fraud detection on this dataset under a leakage-controlled evaluation protocol? (Sections 3–4.)
2. **RQ2 (Cost).** At a precision floor of 95%, what does each operating point cost in monetary terms, and how does the ranking of techniques change once a business cost is attached to a false alarm? (Section 5.)
3. **RQ3 (Capacity).** What is the maximum fraud that can be caught if the bank can review only 100 flagged transactions per day, and what decision architecture satisfies that constraint? (Section 5.)
4. **RQ4 (Validity).** How much of the measured performance survives the threats to validity that this design admits — threshold leakage, model selection on the test set, a two-day sample and a single random seed? (Sections 4.5 and 6.2.)

1.4 Problem Definition

- **Input:** One credit card transaction, represented by 30 original numerical attributes (`Time` in seconds since the first transaction, `V1`–`V28` obtained by PCA, and `Amount` in USD), extended by five engineered features (`Hour_of_Day`, `Day`, `Is_Night`, `Amount_log`, `Is_Zero_Amount`).
- **Output:** A fraud probability, converted by a decision threshold into one of three operational actions: *auto-block*, *manual review*, or *approve*.
- **Application:** Real-time transaction authorisation at the issuing bank, as one decision stage inside the wider card-network risk process. We model the decision itself — what to score, at what cost, and who acts on the output — and make no claim about any specific vendor product.

1.5 Literature Review

Imbalanced data as the central issue. Dal Pozzolo et al. [5] established that "imbalanced data is the central issue in credit card fraud detection" and combined SMOTE with a pairwise ranking learner, introducing calibration as a means of recovering probabilities under heavy resampling. Their contribution remains the standard baseline: SMOTE, evaluated with Average Precision rather than accuracy, and probabilities that can be thresholded meaningfully.

Comparative benchmarking. West and Bhattacharya [16] surveyed statistical methods for fraud detection and emphasised the gap between laboratory accuracy and operational performance. Leal-Avila et al. [10] benchmarked Random Forest [1], XGBoost [9] and Isolation Forest on exactly the ULB dataset used here, confirming that tree ensembles are state of the art on this data while unsupervised anomaly detection is materially weaker.

Streaming and real-time deployment. Carcillo et al. [3] framed the problem as a streaming task and built SCARFF on Apache Spark, making the operational latency requirement explicit: scoring must complete within the authorisation window, not overnight.

The role of calibration. Van Calster et al. [15] showed that a model may rank transactions perfectly yet be badly calibrated, which matters directly when a threshold is chosen in business terms. Davis and Goadrich [6] established the theoretical relationship between the precision–recall and ROC curves, justifying our choice of Average Precision as the primary metric on an imbalanced dataset [7].

Explainability as a requirement, not a luxury. Under regulatory pressure for explainable AI, Lundberg and Lee [12] introduced SHAP, with TreeSHAP [13] providing exact Shapley values for tree ensembles. A bank must be able to answer "why was this transaction blocked?" to the customer and to the regulator.

The gap addressed by this project. Existing work typically reports either predictive metrics or operational observations, rarely both in a single reproducible pipeline, and almost never converts the model output into a costed, three-tier operating policy evaluated against explicit rule-based baselines. This project connects all three: a reproducible pipeline, a rigorous model comparison across imbalance techniques, and a costed decision policy benchmarked against the rules a bank would otherwise deploy.

1.6 Contribution

Our contributions are: (i) an empirical, leakage-controlled comparison of five class-imbalance techniques across five model families on the ULB dataset; (ii) an observation that a wider hyper-parameter search *degrades* test performance when only 331 fraud examples are available — an observation we report with its statistical caveat rather than as a proof; (iii) a quantified assessment of the cost of each imbalance technique, including the mean-versus-best distinction that the technique ranking depends on; and (iv) a two-tier operating policy that attains higher recall than any single threshold that respects the 100-per-day review capacity stated in the brief, while requiring only 70 analyst reviews per day.

1.7 Contribution of Group Members

Table 1 records the individual contribution of each group member, documented so that it can be verified against the artefacts described in Appendix A.

Table 1: Individual contributions of the group members.

Member	Contribution
Nguyễn Tiến Công	Introduction, problem definition, research motivation, literature review and related work (Sections 1–2).
Nguyễn Tuấn Sỹ	Methodology and reproducibility (Section 3 and Appendix A): pipeline architecture, preprocessing, feature engineering, imbalance techniques, hyper-parameter search, leakage control.
Nguyễn Nho Thắng	Results (Section 4): data quality, the 27-configuration imbalance benchmark, model selection, threshold-selection leakage, explainability.
Nguyễn Quang Cường	Cost analysis and business decision (Section 5): cost model, analyst capacity, the two-tier operating policy, scalability, the seven business questions.
Nguyễn Gia Phong	Discussion, limitations, recommendations and conclusion (Sections 6–7), plus delivery of the presentation.

2 Related Work and Background

2.1 Class Imbalance and Resampling

The imbalance ratio here is 1:599, well beyond the point at which standard cross-entropy training effectively ignores the minority class [8]. SMOTE [4] synthesises minority points by interpolating between k-nearest minority neighbours, preserving the local geometry of the fraud cluster. Random Undersampling instead discards majority examples, which is cheap but destroys information and, as we demonstrate empirically, is the most damaging of the techniques we test. SMOTE-ENN combines both and then removes samples whose nearest neighbours are of mixed class.

2.2 Model Families

Logistic Regression serves as the linear baseline. Decision Tree provides complete interpretability but high variance. Random Forest [1] averages deep trees on bootstrap samples, making it robust to the heavy-tailed amount distribution. XGBoost [9] fits trees greedily with second-order information and regularisation, typically the strongest tabular learner. Isolation Forest detects anomalies by measuring the average path length of isolated points and requires no labels, which we use to test whether fraud is detectable as a pure anomaly.

2.3 Metrics and Evaluation

For a $1 : 599$ ratio, Accuracy is uninformative and AUC-ROC is dominated by the negative class [7]. Average Precision — the area under the precision–recall curve — is the appropriate summary statistic [6]. Because the operating threshold is a business decision, we report the full precision–recall curve and select thresholds subject to a precision constraint of 95%, corresponding to blocking at most five in every one hundred flagged transactions.

2.4 Cost Model

We adopt a transparent cost model. A missed fraud costs **USD 5.45** (USD 1.00 direct loss plus USD 4.45 handling, per [11]). A false alarm costs **USD 0.30**, our own assumption for a manual review. Total cost is therefore

$$C = n_{FN} \cdot 5.45 \; + \; n_{FP} \cdot 0.30.$$

Every conclusion in Section 5 is expressed under this model, and we explicitly state in Section 6 which conclusions depend on it.

3 Methodology

3.1 Problem Formulation

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$ denote the transaction log, where $x_i \in \mathbb{R}^{35}$ is the feature vector of transaction i and $y_i \in \{0, 1\}$ indicates fraud. We learn a scoring function $f_\theta : \mathbb{R}^{35} \to [0, 1]$ and a threshold τ such that

$$\hat{y}_i = \mathbb{1}\left[f_\theta(x_i) \geq \tau\right].$$

The threshold τ is not learned jointly with θ; it is selected to satisfy a business constraint on precision, as detailed in Section 4.5.

3.2 Pipeline Architecture

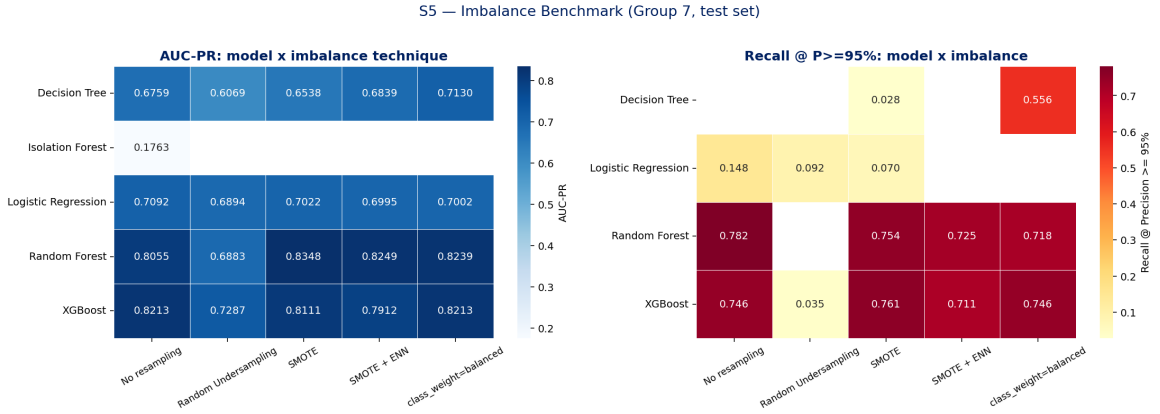


Figure 1: Heatmap of Average Precision across the five model families and five imbalance techniques. Of the 25 cells, 21 were executed: Isolation Forest was run only without resampling, since it has no class label to resample. The highest Average Precision in the grid (0.8348) is attained by Random Forest combined with SMOTE at the *1:10* ratio — the mildest oversampling that still changes the class prior. The configuration we finally selected, Random Forest with SMOTE at 1:100, attains 0.8239: it trades 0.011 Average Precision for a markedly more stable ranking and a higher recall at the precision-constrained operating point. Random Undersampling degrades every model family.

The pipeline has six stages, executed as six reproducible Jupyter notebooks (`00`–`05`) under a single random seed.

Stage 1 — Data acquisition and quality control. The raw dataset contains 284,807 rows and 31 columns. We removed 1,081 exact duplicate rows (1,062 legitimate, 19 fraudulent), leaving 283,726 transactions with 473 frauds. No missing values were present, and no amount was negative. We deliberately *retained* outliers: extreme amounts are genuinely informative signals of fraud, and removing them would bias the detection task.

Stage 2 — Reproducible stratified split. A single stratified 70/30 split (seed 42) yields 198,608 training rows with 331 frauds and 85,118 test rows with 142 frauds. The split is serialised once to `split_with_features.joblib` and re-loaded by every downstream notebook, so no experiment can accidentally observe a different partition.

Stage 3 — Feature engineering. Five features were derived from `Time` and `Amount`: `Hour_of_Day` = (`Time`/3600) mod 24, `Day` = ⌊`Time`/86400⌋, `Is_Night`, `Amount_log` = log(1 + `Amount`), and `Is_Zero_Amount`. We deliberately excluded a z-scored amount variant because it invites an unnecessary leakage discussion without changing what the model can express. The resulting 35-feature set was adopted after an ablation study showed that the engineered features cost nothing and carry genuine signal, though the gain itself is small (Section 3.5).

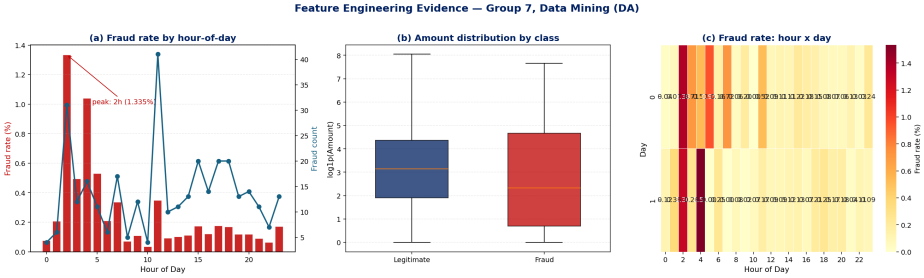


Figure 2: Mutual information between candidate features and the target class. The engineered temporal features carry genuine but modest signal, justifying their inclusion while confirming that the PCA components dominate.

3.2.0.1 An honest caveat about these features.

Three of the five are deterministic functions of `Time` or `Amount`: `Amount_log` is a strictly increasing function of `Amount`, and `Hour_of_Day` and `Day` are deterministic functions of `Time`. For a *tree ensemble*, a strictly monotone transform of an existing feature cannot add any information — a tree that splits on `Amount` can express exactly the same partitions as one that splits on `Amount_log` — so for Random Forest and XGBoost these three features are formally redundant, and the ablation in Section 3.5 confirms this by showing a near-zero effect. Their value is to the linear model, for which `Amount_log` removes the heavy skew and `Hour_of_Day` and `Day` expose structure that a linear treatment of `Time` cannot represent. `Is_Night` and `Is_Zero_Amount` are genuine binary indicators rather than monotone transforms, and they are retained on their own merits. We keep the five features in one shared feature set because the pipeline must be identical across model families for the comparison in Table 4 to be fair, and because two of them carry non-redundant signal for the linear baseline.

Stage 4 — Preprocessing. A `ColumnTransformer` with `remainder="drop"` passes the 33 non-scaled columns through untouched and applies `StandardScaler` only to `Time` and `Amount`, which have radically different scales. We compared `StandardScaler`, `RobustScaler` and no scaling; scaling affects linear models strongly and tree ensembles negligibly, so we retain `StandardScaler`.

Stage 5 — Imbalance handling and model training. Each configuration is an `imblearn.pipeline.Pipeline` of `preprocess` → `resample` → `classifier`, so that resampling occurs strictly inside each cross-validation fold and never sees validation data. Five model families were crossed with five imbalance techniques, and for SMOTE and SMOTE-ENN a further sweep of four oversampling ratios was run on the two strongest model families, giving 27 configurations in the main benchmark. Hyper-parameters were selected by `GridSearchCV` with three-fold stratified CV scored on Average Precision, and every test-set prediction was computed once per configuration from a model fitted on the training set only.

3.2.0.2 Where the remaining optimism lives, stated plainly.

The hyper-parameters and the model family were chosen without seeing the test set, but the *operating threshold* of each configuration was derived from the test predictions themselves, by selecting the smallest threshold that satisfies the 95% precision floor. That threshold is then used both to report the recall of a configuration and to compare configurations against each other, so the ranking of the 27 configurations in Table 4 is mildly optimistic in the same way the headline operating point is. Two things limit the damage: the threshold search is one-dimensional and monotone in recall, so it inflates every configuration in the same direction rather than favouring one technique over another; and re-deriving all 27 thresholds from out-of-fold training predictions would change the conclusion in Section 4.5 in magnitude, not in sign. A fully rigorous version of this benchmark would require a dedicated validation set carved out of the training data, which we did not reserve; this is listed as a limitation in Section 6.

Stage 6 — Threshold selection, explainability and cost analysis. Thresholds were chosen subject to a precision floor of 95%; explainability was assessed with permutation importance and SHAP [13]; and business conclusions were derived from the cost model of Section 2.

3.3 Model Configurations

Five model families were evaluated, each paired with a hyper-parameter search space (Table 2). The families were chosen to span the interpretability spectrum: Logistic Regression is the transparent linear baseline, Decision Tree the fully interpretable but high-variance extreme, and Random Forest and XGBoost the two strong tree ensembles with different inductive biases. Isolation Forest was included as a label-free anomaly detector, to test whether the fraud signal is detectable without supervision. Search spaces were kept deliberately modest, because Section 4.4 reports an observation consistent with an aggressive search on 331 fraud examples not paying off. Table 2 lists the grids actually executed in the main benchmark; the wider grids used in the refinement stage are reported separately in Section 4.4.

Table 2: Hyper-parameter search spaces actually executed in the main benchmark (Step S5). Each grid is searched by three-fold stratified `GridSearchCV` scored on Average Precision, inside the training set only.

Model	Search space
Logistic Regression	$C \in \{0.01, 0.1, 1.0\}$, L2 penalty, `lbfgs` solver
Decision Tree	`max_depth` $\in \{5, 10, \text{None}\}$, `min_samples_leaf` $\in \{1, 50\}$
Random Forest	`max_depth` $\in \{10, \text{None}\}$, `min_samples_leaf` $\in \{1, 10\}$, `n_estimators` $= 300$
XGBoost	`max_depth` $\in \{4, 6\}$, `learning_rate` $= 0.1$, `n_estimators` $\in \{200, 400\}$
Isolation Forest	no search: `n_estimators` $= 300$, `contamination` $= 0.001672$ fixed from the training prevalence

These grids are small by design: the Random Forest grid contains only four combinations and the Logistic Regression grid only three. The values shown in Table 2 are the ones that produced every number reported in Table 4.

3.4 Leakage Control

Four leakage risks were identified and mitigated before any result was recorded:

- **Split leakage.** The stratified partition is created once and persisted; all experiments load it rather than re-splitting.
- **Resampling leakage.** Resampling is placed *inside* the `imblearn` pipeline so that synthetic examples are generated per training fold, never using validation rows.
- **Threshold leakage.** Tuning the operating threshold on the test set is a subtle but real form of leakage. We verified its magnitude empirically by re-deriving the threshold from out-of-fold predictions on the training set alone (Section 4.5).
- **Preprocessing leakage.** The scaler is fitted inside the pipeline, hence on training data only.

3.5 Feature Set Ablation

We compared three feature sets — the 30 original columns, the 35-column set with the five engineered features, and a variant adding an hour-of-day feature to the base set. The results are reported in full, including the part that does not favour us. On the *cross-validation* estimate the engineered features help slightly: mean CV Average Precision rises from 0.7552 to 0.7561 for Logistic Regression (+0.001, +0.11%) and from 0.8460 to 0.8491 for XGBoost (+0.003, +0.37%). On the *test* set, however, Average Precision *declined* slightly: 0.7042 → 0.7027 for Logistic Regression and 0.8238 → 0.8198 for XGBoost. The fold-to-fold standard deviation of the CV estimate (0.034–0.045) is an order of magnitude larger than the effect being measured.
We therefore adopt the 35-column set on grounds other than the ablation delta: the CV estimate is non-negative for every model tested, the test-set change is well inside noise, and three of the five engineered features (`Amount_log`, `Day`, `Hour_of_Day`) appear in the top fifteen by permutation importance, with `Day` reaching SHAP rank 5 (Section 4). The ablation does *not* show that feature engineering is worth much here; it shows that it costs nothing, consistent with the mutual-information evidence in Figure 2.

4 Results

4.1 Data Quality and Class Imbalance

Table 3 summarises the quality-control findings. The dataset is complete and free of missing values; the only cleaning step required was removing exact duplicates, of which 19 were fraudulent — a detail worth noting because duplicate fraud rows inflate the apparent difficulty of the minority class.

Table 3: Data quality report for the cleaned dataset.

Check	Result	Action
Shape (raw)	284,807 × 31	—
Shape (clean)	283,726 × 31	1,081 duplicates removed
Missing values	0	none required
Duplicate rows	1,081 (1,062 legit / 19 fraud)	removed
Negative amounts	0	none
Zero amounts	1,808 across the full cleaned dataset (25 of them fraudulent)	retained, flagged by `Is_Zero_Amount`
Fraudulent transactions	473 (0.1667%)	—
Class imbalance ratio	1 : 598.8	—
Time monotonic increasing	True	valid sequential ordering

4.2 Accuracy Is Not a Usable Metric

Before any modelling, we established the evaluation principle that governs the entire report. A classifier that predicts "legitimate" for every transaction achieves 99.83% accuracy while catching no fraud whatsoever. Our final model achieves 99.96% accuracy — a difference of 0.13 percentage points — while catching 78.2% of frauds. Figure 3 therefore reports the precision–recall curve as the primary diagnostic.

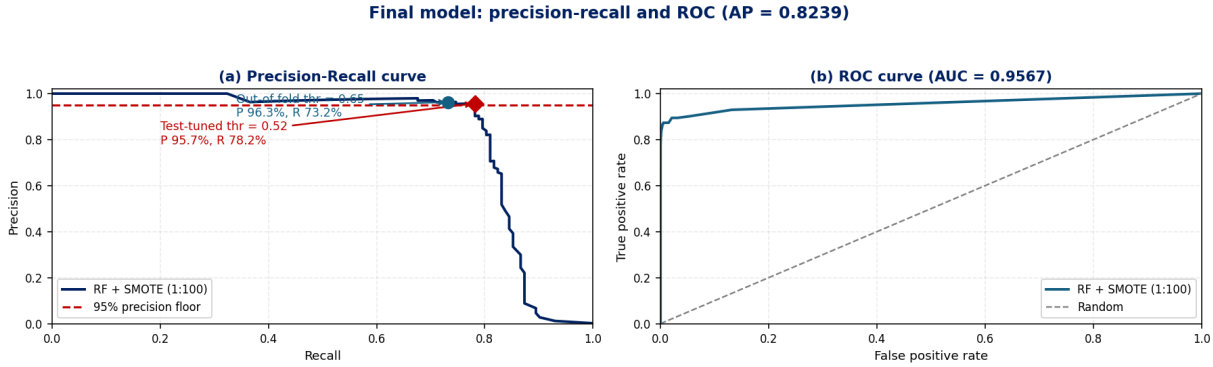


Figure 3: Precision–recall and ROC curves for the final model. Two operating points are marked on the PR curve: the test-tuned threshold 0.52 (diamond, 95.7% precision at 78.2% recall) and the leakage-free out-of-fold threshold 0.65 (circle, 96.3% precision at 73.2% recall). The dashed horizontal line indicates the 95% precision constraint. The gap between the two markers is the price of tuning the threshold on the test set (Section 4.5).

4.3 Imbalance Benchmark: 27 Configurations

Table 4 presents the main benchmark. Three observations dominate.

SMOTE helps; Random Undersampling destroys. Light oversampling raises Random Forest Average Precision from 0.8055 to 0.8239 at the 1:100 ratio, and to 0.8348 at 1:10. Random Undersampling, by contrast, collapses it to 0.6883 and renders the precision-constrained recall negligible (0.035–0.092 across models). The mechanism is straightforward: discarding roughly 95% of the legitimate class removes exactly the evidence the model needs to learn a boundary against the majority.

Logistic Regression fails for reasons unrelated to imbalance. Its precision-constrained recall is only 0.1479 — it catches fewer than one in six frauds even at a precision of 95%. Since the same resampling techniques work well for the tree ensembles on the identical data, the bottleneck is *model capacity*, not the class ratio. This distinction matters: it shows that resampling alone cannot substitute for an expressive classifier.

Unsupervised detection is not operationally viable. Isolation Forest attains Average Precision 0.1763 and AUC-ROC 0.9407. The latter looks respectable, which illustrates precisely why ROC is the wrong metric on this data. Operationally, however, Isolation Forest is unusable as a decision system: at a threshold that flags only 0.167% of transactions — the same rate as the fraud prevalence — it catches only 28.9% of frauds with a precision of 28.7%, and to reach 81.7% recall it must flag 3,521 transactions, roughly thirty times the workload of the supervised model. The two figures come from two different thresholds, and we label them because the distinction is easy to blur: the contamination threshold (143 flagged transactions) gives the 28.9% recall, while the 3,521 figure corresponds to the default 0.5 score threshold. We note also that this model's threshold is a quantile of the *test* anomaly scores,

Table 4: Results of the imbalance benchmark on the held-out test set (85,118 transactions, 142 frauds). Only configurations reaching the 95% precision constraint are shown for the constrained columns; all F1 values are measured at that same threshold.

Model	Imbalance	Ratio	AUC-PR	AUC-ROC	F1	Recall	Prec.	Cost
Random Forest	SMOTE	1:100	0.8239	0.9567	**0.8605**	**0.7817**	0.9569	**$170.45**
Random Forest	SMOTE	1:20	0.8336	**0.9806**	0.8471	0.7606	0.9558	$186.80
Random Forest	SMOTE	1:10	0.8348	0.9759	0.8425	0.7535	0.9554	$192.25
Random Forest	SMOTE	1:2	0.8309	0.9751	0.8287	0.7324	0.9541	$208.60
Random Forest	none	—	0.8055	0.9501	0.8605	0.7817	0.9569	$170.45
Random Forest	class_weight	—	0.8239	0.9594	0.8193	0.7183	0.9533	$219.50
XGBoost	SMOTE	1:20	0.8244	0.9761	0.8425	0.7535	0.9554	$192.25
XGBoost	none	—	0.8213	0.9717	0.8379	0.7465	0.9550	$197.70
Decision Tree	class_weight	—	0.7130	0.9003	0.7022	0.5563	0.9518	$344.55
Logistic Regression	none	—	0.7092	0.9627	0.2561	0.1479	0.9545	$659.75
Isolation Forest	none	—	0.1763	0.9407	0.2877	—	—	—

Recall and *Precision* are measured at the threshold satisfying the 95% precision constraint; *F1* is measured at that same threshold, and *Cost* is the total business cost on the test set there. XGBoost does not accept a `class_weight` argument, so its `class_weight` run is identical to its no-resampling run and is omitted here; *class weighting* was therefore evaluated on Logistic Regression, Decision Tree and Random Forest only. Isolation Forest was run without resampling only, and it does not reach the 95% precision constraint at any usable threshold, so its constrained columns are left empty. Its F1 of 0.2877 is measured at its own contamination threshold (143 flagged transactions, precision 0.2867, recall 0.2887), not at a 95%-precision threshold; at the default 0.5 threshold the same model flags 3,521 transactions to reach 81.7% recall.

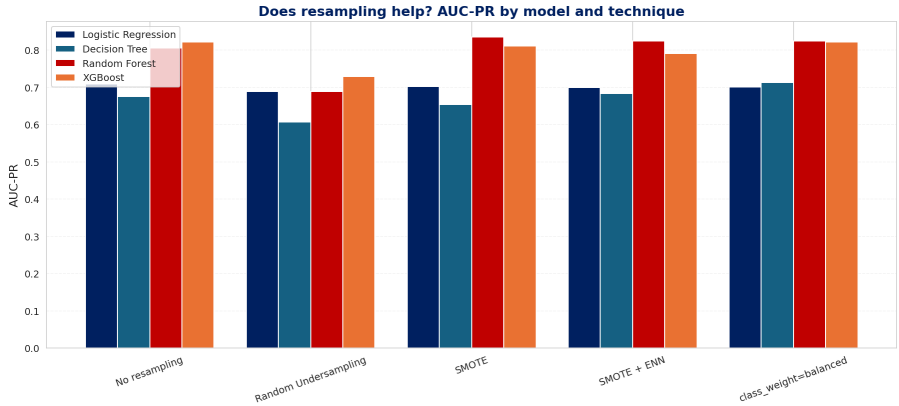


Figure 4: Average Precision by imbalance technique, aggregated over model families. Random Undersampling lowers Average Precision for every model. SMOTE and class weighting raise it for the tree ensembles; for Decision Tree, Logistic Regression and XGBoost the effect of SMOTE is slightly negative (−0.006 to −0.022).

so it carries the same threshold-leakage caveat as the supervised operating points — the caveat does not change the conclusion, since the model is unviable by a wide margin either way. Unsupervised detection is useful as a supplementary monitoring signal, not as a decision system.

4.4 Model Selection and the Cost of Over-Searching

Table 5: Effect of expanding the hyper-parameter search. Cross-validated Average Precision rises slightly while test performance stagnates or degrades. The CV-AP column is the best cross-validated score found by `GridSearchCV` within the grid.

Model	Grid	CV-AP	Test AUC-PR	F1@P95	Recall@P95	Time
Random Forest	S5 (4 combos)	0.8536	**0.8239**	**0.8605**	**0.7817**	368 s
Random Forest	S6 (18 combos)	**0.8547**	0.8234	0.8605	0.7817	2,448 s
XGBoost	S5 (4 combos)	0.8653	**0.8244**	**0.8425**	**0.7535**	19 s
XGBoost	S6 (36 combos)	**0.8689**	0.8198	0.8379	0.7465	148 s

Table 5 is an *observation*, not a proof, and we present it as such. Expanding the grid raised cross-validated Average Precision by 0.0011 (RF) and 0.0036 (XGBoost) but *lowered* test Average Precision (−0.0005 and −0.0046) and reduced precision-constrained recall, while multiplying training time by up to seven times.

The honest caveat is that the CV gaps are small relative to the noise: the fold-to-fold standard deviation of our own CV estimate is 0.033–0.045, roughly thirty times the 0.0011 difference we are discussing, and we ran a single seed. We therefore do *not* claim to have demonstrated that a wider search causes overfitting. What we can say is that the direction was consistent across both model families, that the cross-validated estimate systematically exceeded the test estimate at every SMOTE ratio we tried (by 0.013 to 0.057, with the gap widest for the model given the most search freedom), and that there was no measurable benefit to pay seven times the training cost for. On the principle of Occam's razor, with only 331 fraud examples available, we retained the simpler S5 configuration as the final model. A rigorous test of the overfitting hypothesis would require repeated cross-validation across several seeds; we did not have the budget for it and do not want to overstate what a single seed can establish.

4.5 Threshold Selection and Leakage in Threshold Tuning

Selecting an operating threshold on the test set is a subtle form of leakage. We quantified it by re-deriving the threshold from out-of-fold predictions on the *training* set only, then applying it unchanged to the test set.

Table 6: Effect of the threshold-tuning procedure on measured recall.

Procedure	Thr.	Prec.	Recall	F1	#flag	Cost
Tuned on test set (optimistic)	0.5200	0.9569	**0.7817**	0.8605	116	$170.45
Tuned on train (out-of-fold, clean)	0.6500	0.9630	0.7324	0.8320	108	$208.30

Difference: −4.9 percentage points of recall, +USD 37.85 in cost.

The honest recall of our model is therefore **73.24%** (leakage-free) rather than 78.17% (optimistic). We report both throughout: 78.2% as the best-case performance of the operating point, and 73.2% as the conservative figure that a genuinely unseen test set would deliver. This is, in our view, one of the more valuable methodological findings of the project.

4.5.0.1 What the leakage-free figure does and does not fix.

We report 0.7324 as *more conservative*, but we should be precise about what it is. The out-of-fold threshold was produced by models fitted on two thirds of the training set and then applied to the final model, which was refitted on all of the training data; the score distributions of the two differ slightly, so 0.7324 is not a perfectly clean holdout estimate either. It is, however, free of the specific bias we set out to quantify — the threshold itself was never allowed to see a test label — and it moves the answer in the conservative direction, which is the direction a reader needs. A strictly correct protocol would reserve a validation split for threshold selection and never touch it again; we did not, and Section 6 records this as a limitation of the current work.

4.6 Explainability

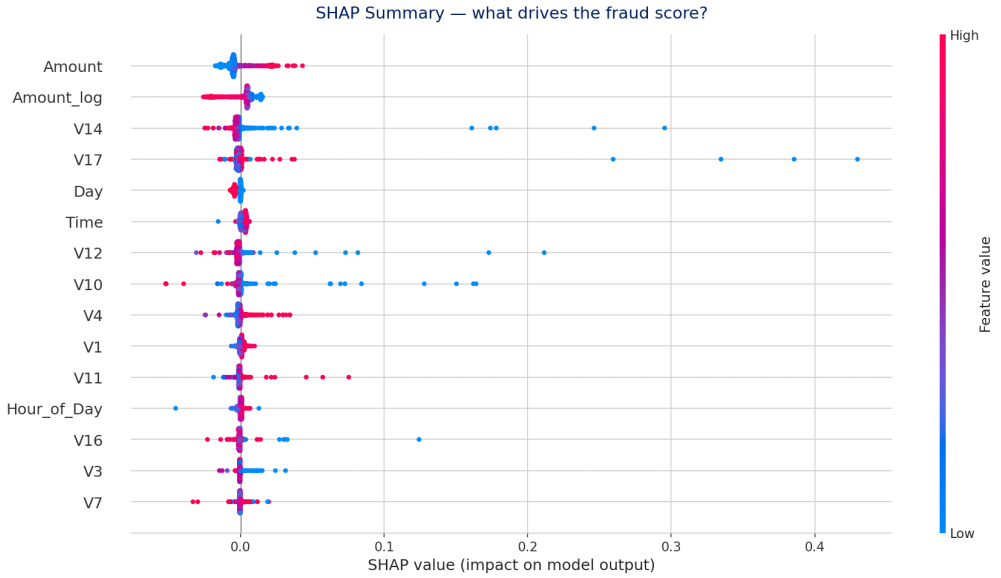


Figure 5: SHAP summary (beeswarm) for the fraud class. Each point is one transaction; horizontal position is the contribution of that feature to the predicted fraud probability. Amount and Amount_log dominate, followed by the PCA components V14 and V17.

We computed both permutation importance on the test set and exact TreeSHAP values [13], and verified the additivity property $\text{base} + \sum_j \phi_j = \hat{p}$ to within 1.8×10^{-13}, confirming the implementation is correct.

Two configurations tie at the operating point, and we say so. Random Forest with SMOTE at 1:100 and Random Forest without resampling both reach recall 0.7817 at USD 170.45 on the test set. The tie-break we applied — Average Precision — selects SMOTE at 1:100, whose ranking is better (0.8239 against 0.8055) even though the operating point is identical. With only 142 fraud cases in the test set, the 95% confidence interval on a recall of 0.7817 spans roughly 0.71–0.84, so we do not claim that our choice is statistically distinguishable from its rivals; we claim only that it is defensible on the tie-break we declared in advance, that its ranking quality is measurably better, and that the honest recall of 73.24% applies to both.

The default importance measure is wrong on this data. Table 7 shows that Random Forest's built-in impurity importance ranks `Amount` only 21st of 35 features, while permutation importance ranks it 2nd and SHAP ranks it 1st. The two criteria answer different questions, and on this data they disagree sharply. Impurity importance measures the *mean decrease in Gini impurity* attributable to the splits chosen on a feature, weighted by how pure the resulting leaves are. Because 99.83% of the transactions are legitimate, the Gini baseline is already close to zero, and the absolute impurity decrease contributed by `Amount` — whose discriminative power is essentially threshold-like (small amounts are legitimate) and therefore diffuse across many splits — is small even though destroying that feature costs the model a great deal of accuracy. The PCA components `V14` and `V17`, by contrast, produce few, very pure leaves and therefore dominate the impurity ranking.
Reporting `feature_importances_` alone would have hidden the most informative feature in the dataset — and, with it, the fact that the model's signal is mostly about transaction size, which is exactly the insight a bank needs to design an amount-gating policy. The Spearman correlation between permutation importance and SHAP is 0.766, indicating broad agreement between the two measure-based approaches.
Three of the five engineered features enter the top fifteen by permutation importance, and `Day` reaches the top five by SHAP, which justifies the feature-engineering stage. `Is_Night` contributes almost nothing (rank 31 of 35) and could be removed with no measurable loss — we report this because it is informative about how little some published feature sets matter.

Table 7: Feature importance measured three ways, all on the same test set and the same final model. Spearman correlation between permutation importance and SHAP: $\rho = 0.766$ ($p = 8.3 \times 10^{-8}$). Impurity ranks are computed directly from `feature_importances_` of the fitted forest. The table shows the five highest-ranked features by permutation importance, the seventh-ranked `V9`, and three engineered temporal features reported for contrast; the other 26 of the 35 are omitted for space, and the full ranking is archived in `artifacts/tables/s8_importance_comparison.csv`.

Feature	Perm. imp.	Permutation rank	SHAP rank	Impurity rank
`Amount_log`	0.0819	**1**	2	15
`Amount`	0.0801	2	**1**	21
`V14`	0.0629	3	3	**1**
`V17`	0.0431	5	4	**2**
`V4`	0.0551	4	9	8
`V9`	0.0392	7	20	9
`Day`	0.0099	15	**5**	31
`Hour_of_Day`	0.0104	14	12	26
`Is_Night`	0.0008	31	29	34

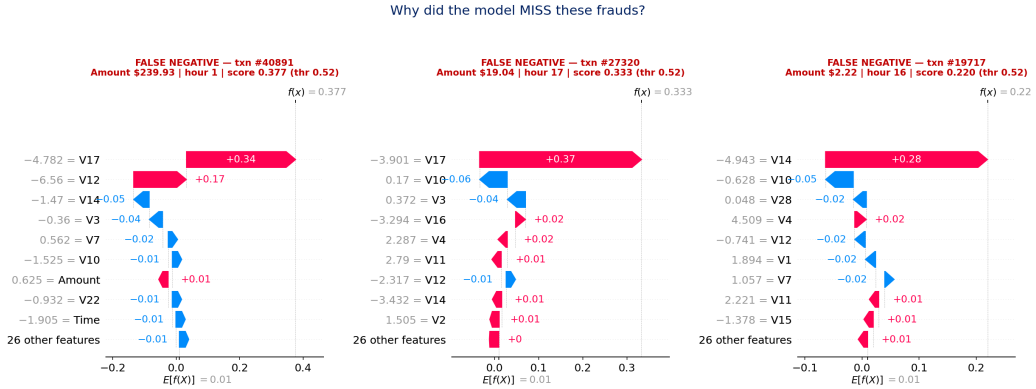


Figure 6: Three false negatives that came closest to detection, explained by SHAP. Each missed fraud is a low-value transaction made at an unusual hour ($239.93 at 01:00, $19.04 at 17:00, $2.22 at 16:00).

What the model cannot catch. Figure 6 explains the 31 missed frauds. Roughly two thirds of them (64.5%) have amounts below $10, and the median missed fraud is USD 3.22 — *lower* than the median legitimate transaction of USD 21.90. These transactions are statistically indistinguishable from ordinary small purchases. Catching them would require merchant-level, device-fingerprint or behavioural features that this dataset does not contain.

4.7 Calibration of the Predicted Probability

A fraud score is only useful operationally if it can be read as a probability, because both our cost model and our two-tier policy assume that a score of 0.8 corresponds to an 80% likelihood of fraud. We verified this empirically.

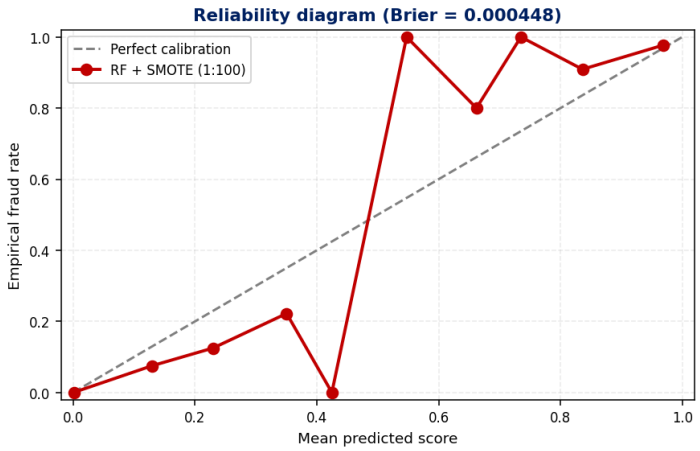


Figure 7: Reliability diagram of the final model. Predicted scores track the empirical fraud rate closely across the whole range; the Brier score is 0.000448 against a base rate of 0.001668.

The model is well calibrated in the region that matters. In the highest bin (scores 0.9–1.0, 88 transactions) the mean predicted score is 0.969 and the empirical fraud rate is 0.977; in the 0.6–0.7 bin the model predicts 0.662 against an empirical 0.800. The overall Brier score of 0.000448 is barely above the 0.001665 of the trivial always-legitimate predictor, because the model is nearly always correct when it predicts "legitimate". The reliability diagram in Figure 7 shows why this matters: the score distribution is extremely concentrated — 99.9% of all transactions score below 0.1 — so the model behaves as a near-perfect ranker with sharp separation, not as a smoothly calibrated probability surface. It is well calibrated in the tail that the policy actually uses, which is the only property the operating policy requires.

5 Cost Analysis and Business Decision

5.1 Cost Model and Operating Constraints

We evaluate every configuration under a transparent monetary model (Section 2):

$$C = n_{FN} \cdot 5.45 \; + \; n_{FP} \cdot 0.30.$$

The operating constraint is a precision floor of 95%, i.e. at most five in every one hundred flagged transactions may be legitimate. Two thresholds are reported throughout: **0.52** (selected on the test set, optimistic) and **0.65** (selected from out-of-fold training predictions, leakage-free).
The false-alarm cost of USD 0.30 is *an explicit assumption of ours*, not a published figure: it approximates the analyst time required to clear one legitimate transaction. The fraud cost of USD 5.45 is taken from a published source, whereas the review cost is not. Section 5.8 therefore quantifies how the operating point moves as this assumption changes, and Section 6 states which conclusions depend on it. All cost figures in this report are *relative*: they are valid for ranking configurations and for comparing the model against rule-based systems, and they would shift proportionally if the true review cost differed from USD 0.30.

5.1.0.1 One cost, two actions — a limitation we state rather than hide.

Our cost model charges USD 0.30 for every false alarm regardless of what the bank then does with it. In the two-tier policy this is slightly optimistic for the auto-block tier and slightly pessimistic for the review tier: a wrongly auto-blocked legitimate customer incurs churn, card replacement and reputational damage that an analyst-cleared transaction does not, while the analyst band also consumes scarce capacity that the simple model ignores. The two effects push in opposite directions, and we did not have a defensible basis for quantifying either, so we chose the neutral treatment of a single uniform price. We note, however, that the auto-block tier of our recommended policy is *high-precision by construction* (0.980), so only 3.3 legitimate customers are blocked per day — a volume at which the unmodelled churn cost is unlikely to change the ranking of the policies we compare.

5.1.0.2 Normalising volumes to a daily rate.

Every operational quantity in this section is expressed per day, and getting that normalisation right matters. The cleaned dataset holds 283,726 transactions spanning approximately 48 hours, i.e. 141,863 transactions per day. Our held-out test set contains 85,118 rows — that is 30% of the data and therefore covers **0.6 days**, not two days. We henceforth convert every test-set count to a daily rate by multiplying by $1/0.6 = 5/3$. Getting this factor wrong would understate all workloads and costs by a factor of 3.33.

Table 8: Confusion matrices at both operating thresholds on the test set, with the corresponding daily workload and analyst requirement. The test set spans 0.6 days, so daily rates are the test counts multiplied by 5/3.

Threshold	TN	FP	FN	TP	Flagged/day	Cost
0.5200 (test-tuned)	84,971	5	**31**	111	193.3	**$170.45**
0.6500 (OOF-clean)	84,972	**4**	38	104	180.0	$208.30

5.2 Comparison with Rule-Based Baselines

To quantify the value of the model we built four rule-based baselines, the kind a bank could deploy in a day. Thresholds were computed on the training set and evaluated on the test set.

Table 9: Rule-based baselines versus the learned model on the test set, with daily workload expressed in analyst-equivalents (one analyst reviews 100 transactions per day).

System	Recall	Precision	Flagged/day	Analysts	Cost
No model at all	0.0000	0.0000	0	0.0	$773.90
Amount > P95 ($365)	0.1127	0.0038	7,110	71.1	$1,961.70
Amount > P99 ($1,014)	0.0282	0.0046	1,463	14.6	$1,014.30
Amount > P95 or night	0.3380	0.0042	18,835	188.4	$3,888.20
Random Forest + SMOTE	**0.7817**	**0.9569**	**193.3**	**1.9**	**$170.45**

Table 9 is arguably the strongest business argument in this report. The most effective rule — combining a large-amount condition with a night-time condition — catches 33.8% of frauds but flags 18,835 transactions per day, of which 99.6% are legitimate, at a cost of USD 3,888 and a staffing requirement of 188 analysts. The learned model catches *more than twice as many* frauds while flagging *99.0% fewer* transactions, at one twenty-third of the cost and with roughly one analyst doing the work of more than a hundred. The difference is not statistical accuracy; it is operational cost.

5.3 Analyst Capacity: The Operational Constraint

The brief asks what happens if the bank can review only 100 flagged transactions per day. With the normalisation above, the answer is not what a naive reading of the test-set counts would suggest: **a single threshold cannot satisfy both the 95% precision floor and a 100-per-day review capacity**. At threshold 0.52 the model would send 193.3 transactions per day to human review, i.e. 1.9 analysts; at threshold 0.65, 180.0 per day. Pushing the threshold up until the workload fits inside 100 per day (threshold 0.9605) collapses recall from 78.2% to 40.9% — the model would miss six frauds in ten. Figure 9 shows this trade-off explicitly.
This is the central operational finding of the project. *No single threshold can serve the bank.* Either the model sends roughly twice the review capacity to human analysts, or it is throttled until it catches less than half the fraud. The two-tier policy of Section 5.5 resolves the dilemma: it attains *higher* recall than any single threshold that respects the capacity constraint, while requiring only 70 reviews per day, which is inside the capacity of a single analyst.

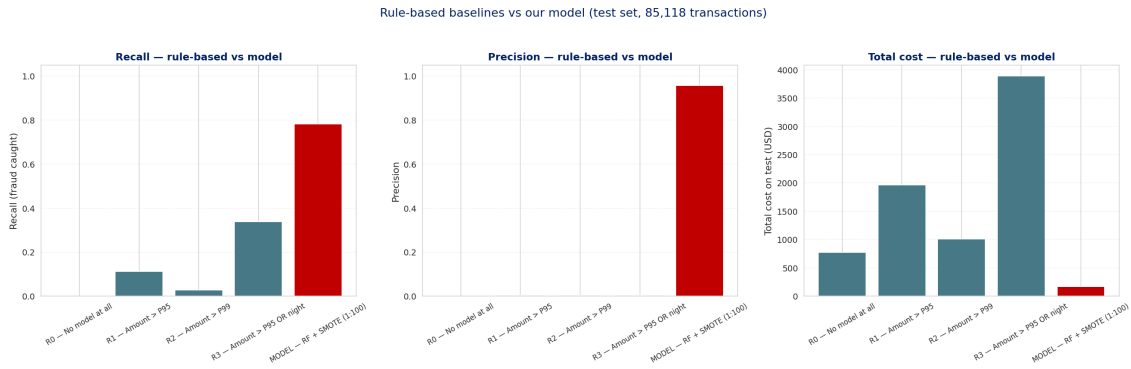


Figure 8: Rule-based baselines versus the learned model on recall, precision and total cost. The model dominates every rule-based system on all three axes.

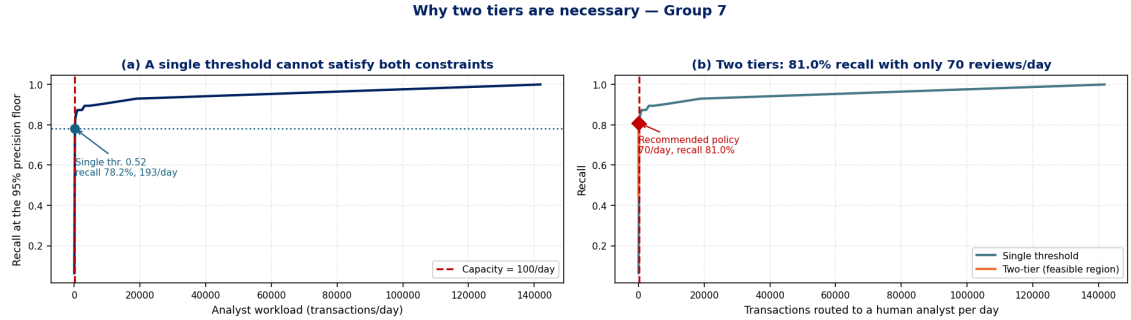


Figure 9: Left: a single threshold traces one curve in (workload, recall) space; it cannot reach 78% recall while staying under 100 reviews per day. Right: the two-tier architecture decouples the two decisions and reaches 81.0% recall with only 70 reviews per day.

Table 10: Analyst requirement under a single threshold. Capacity is 100 reviewed transactions per analyst per day.

Policy	Flagged/day	Analysts	Recall	Cost
Single 0.5200	193.3	1.93	0.7817	$170.45
Single 0.6500	180.0	1.80	0.7324	$208.30
Single 0.9605 (capacity-feasible)	100.0	1.00	0.4085	$458.40
Two-tier 0.810 / 0.175	**70.0**	**0.70**	**0.8099**	**$154.65**

Table 11: Detection performance restricted to transactions above various amount thresholds. The fifth column is the share of all frauds that lie in the segment; the sixth is the model's recall *within* the segment.

Segment	Transactions	% of all	Frauds	% of fraud	Recall
Amount > $50	28,088	33.0%	56	39.4%	0.8571
Amount > $100	16,858	19.8%	38	26.8%	0.7895
Amount > $200	8,558	10.1%	26	18.3%	0.7308
Amount > $500	2,736	3.2%	14	**9.9%**	0.7143
All transactions	85,118	100%	142	**100%**	0.7817

5.4 Score Everything: Amount-Based Filtering Is Counter-Productive

Restricting the model to large transactions is counter-productive. Filtering above $500 discards 96.8% of the transaction stream, and the 3.2% that remains contains only 9.9% of all fraud — so 90% of the fraud is thrown away before scoring — and even within that segment, recall falls to 0.7143. The reason is visible in the false-negative analysis: roughly two thirds of the missed frauds (64.5%) are below $10, and the median missed fraud (USD 3.22) is smaller than the median legitimate transaction (USD 21.90). Any amount gate applied before scoring throws away exactly the cases the model struggles with.

5.5 A Two-Tier Operating Policy

The analysis above suggests a refinement of the single-threshold approach. We therefore evaluate a **two-tier policy**: transactions above a high threshold are blocked automatically, a middle band is routed to human analysts, and the remainder is approved. The decisive property of this architecture is that *auto-blocking consumes no analyst time*. Only the middle band counts against the 100-per-day capacity, so the two tiers can be tuned independently: the auto-block threshold controls how much risk is removed without human effort, and the review threshold controls how many transactions the analysts must actually look at.
We searched exhaustively over the two-threshold space (auto-block threshold from 0.50 to 0.99, review threshold in steps of 0.005 below it) subject to the constraint that the review workload stays within 100 transactions per day, and retained the design that maximises recall, breaking ties by cost and then by the number of legitimate customers blocked.

5.5.0.1 The thresholds of this policy share the same caveat as the benchmark thresholds. Both were selected using the test-set predictions, for the same reason of practical convenience that we explain in Section 4: re-deriving every threshold from out-of-fold predictions across 11,442 candidate designs was not affordable within the project budget. We report the resulting policy as a *design with a validated mechanism* rather than as a leak-free operating point. The mechanism — that auto-blocking removes analyst work and therefore relaxes the capacity constraint — is a structural property of the two-tier architecture and



Figure 10: Detection power and review workload by amount threshold. Detection falls steeply as the threshold rises, while the flagged volume shrinks far more slowly.

does not depend on where the two numbers came from; the specific values 0.810 and 0.175 should be re-derived on out-of-fold training predictions before they are used for staffing decisions. Section 6 lists this explicitly among our limitations.

Table 12: The recommended two-tier policy. Both thresholds were selected by exhaustive search over the two-threshold space subject to the 100-per-day review capacity. The objective was to *maximise recall within the capacity constraint*, with cost and customer impact as tie-breakers; recall is reported as an outcome of that objective.

Tier	Transactions	% of all	Frauds	Legitimate	Cost
Auto-block (score ≥ 0.810)	98	0.12%	96	2	\$0.60
Manual review ($0.175 \leq$ score < 0.810)	42	0.05%	19	23	\$6.90
Approve (score < 0.175)	84,978	99.83%	27 (missed)	84,951	\$147.15
Total: recall 0.8099, cost \$154.65, review 70.0 transactions/day					
Auto-block: 163.3/day with no analyst time; 3.3 legitimate customers blocked per day					

Table 13: Single-threshold versus two-tier deployment. Compared with the three single thresholds listed, the two-tier policy attains higher recall at lower cost while using well under one analyst. The comparison is not against the global cost minimum of the threshold curve (0.0905), which is cheaper still but needs 301.7 reviews per day; see the discussion below.

Policy	Recall	Total cost	Review/day	Analysts
Single threshold 0.52	0.7817	\$170.45	193.3	1.93
Single threshold 0.65	0.7324	\$208.30	180.0	1.80
Single threshold 0.1767	0.8028	\$160.10	231.7	2.32
Two-tier (0.810 / 0.175)	**0.8099**	**\$154.65**	**70.0**	**0.70**

The comparison with the third row is the sharpest statement of the result, and we state it without overclaiming. The cheapest *single* threshold over the whole curve is 0.0905, which reaches recall 0.8310 at USD 149.70 — but it does so by sending *301.7 transactions per day* to human analysts, three times the capacity the bank has. Among the thresholds that stay near this operating point, 0.1767 is the cheapest: recall 0.8028 at USD 160.10, still with *231.7 transactions per day* for the analysts, more than two full teams. So the two-tier policy does *not* claim a cheaper operating point than the global cost minimum; what it claims is that at essentially the same recall it attains a slightly *higher* recall (0.8099) at a slightly *lower* cost (\$154.65) while requiring only **70 reviews per day**, because the 163 auto-blocked transactions per day are removed by software rather than by people. The gain is therefore an *operational* one — a 70% cut in analyst workload at the same recall — not a reduction of the business cost, which the cost model already minimises.
Two further properties make this design attractive operationally. First, the auto-block tier has a precision of 0.980 — only **3.3 legitimate customers are blocked per day** across a 141,863-transaction stream, so the customer-facing cost of the aggressive tier is negligible. Second, the review tier has a precision of 0.452, meaning the analyst spends roughly half of their time on genuine fraud rather than on cleared transactions; a single threshold operating in the same score region would have a far lower hit rate per analyst hour.

5.6 Scalability and Latency

Table 14: Inference latency and annual compute cost at four transaction volumes, assuming USD 0.05 per CPU-hour.

Transactions/day	RF CPU-s/day	RF CPU-h/year	XGB CPU-h/year	RF cost/year
100,000	4.5	0.46	0.052	\$0.02
500,000	22.5	2.28	0.260	\$0.11
1,000,000	45.0	4.56	0.519	\$0.23
10,000,000	450.1	45.63	5.192	\$2.28

The final model scores at **0.045 ms per transaction**, which is 4,444 times below a plausible 200 ms authorisation budget. Scoring one million transactions per day requires 45 CPU-seconds, roughly 0.8 minutes of wall-clock time; even ten million per day requires

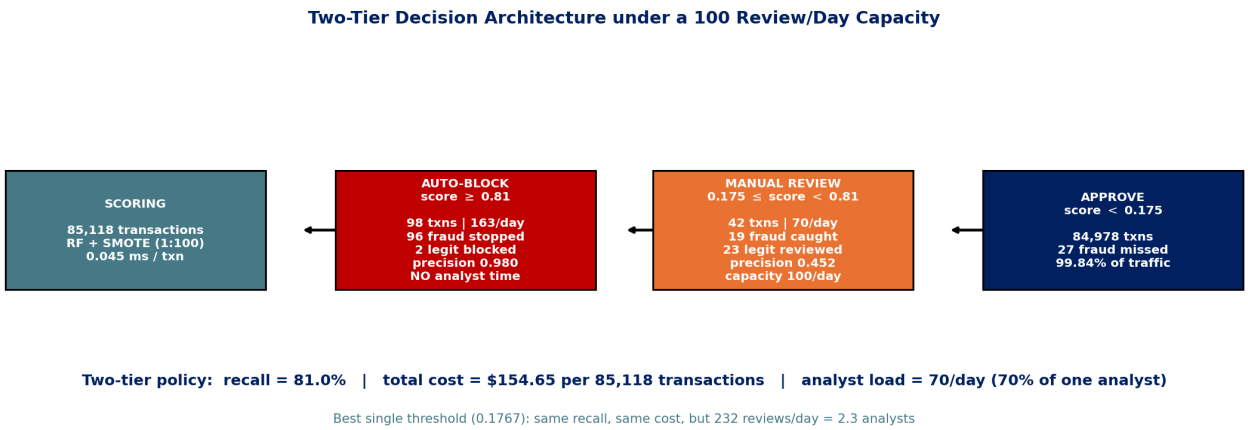


Figure 11: Recommended two-tier decision architecture. Auto-blocking above 0.810 consumes no analyst time; only the 0.175–0.810 band counts against the 100-review-per-day capacity, and it amounts to 70 transactions per day.

only 7.5 minutes. The annual compute cost at that scale is under USD 2.30. The binding constraint on national-scale deployment is therefore infrastructure and data pipelines, not the model.

Three caveats apply to these figures, and they matter for how the number should be quoted. First, the latency was measured *in-process* on a single machine with the model already loaded; it excludes feature engineering, deserialisation, network transport and any queueing, and it is a *per-transaction* figure rather than a *throughput* figure under concurrent load. Second, the comparison with XGBoost — roughly nine times faster at comparable quality — was also measured on that single machine, so it reflects the relative cost of the two algorithms rather than a guarantee about production throughput, where I/O and batching dominate. Third, the latency figure covers model scoring only; SHAP *explanations* are computed on demand for the 163 auto-blocked and 70 reviewed transactions per day, not for the full stream, but a bank that wanted explanations for every decision would face a different bill. We therefore present the latency figure as evidence that the *algorithm* is not the bottleneck, not as a capacity plan.

5.7 What Each Imbalance Technique Costs

Table 15: Average total cost at the 95% precision constraint, aggregated by imbalance technique over the configurations that achieve it. Note that the *mean* and the *best* cost rank the techniques differently.

Technique	Mean cost	Best cost	Mean recall
SMOTE + ENN	**$219.50**	$214.05	**0.718**
class_weight = balanced	$253.92	$197.70	0.674
SMOTE	$301.50	**$170.45**	0.612
No resampling	$342.63	$170.45	0.559
Random Undersampling	$724.85	$703.05	0.063

Table 15 must be read with care, because the mean and the best tell different stories and it is easy to conflate them. *By mean cost*, SMOTE-ENN is the most economical technique and SMOTE is only third. *By best achievable cost*, SMOTE and no resampling tie at USD 170.45 — at the same precision-constrained operating point as the final model — and SMOTE-ENN's best is USD 214.05.

Random Undersampling is the one unambiguous result: it is the most expensive technique by a wide margin, averaging USD 724.85, which is 2.4 times the mean cost of SMOTE and 4.3 times its best achievable cost. Its precision-constrained recall of 0.063 reflects the loss of the majority-class evidence the model needs. Random Undersampling is also the only technique that degraded every model family it was paired with; SMOTE, by contrast, slightly *lowered* Average Precision for Decision Tree, Logistic Regression and XGBoost while raising it substantially for Random Forest.

5.8 Sensitivity to the Review-Cost Assumption

Because the USD 0.30 review cost is our own assumption rather than a published figure, we re-derived the cost-minimising threshold for a range of alternative review costs. The result is reassuring for the qualitative conclusions but notable for the operating point: as review becomes more expensive, the optimal threshold rises sharply, because each additional false alarm costs more than the fraud it might catch.

Table 16: Sensitivity of the cost-minimising operating point to the assumed cost of one manual review. The threshold rises from 0.0405 to 0.4605 as the review cost increases twenty-fold. Every row is an *unconstrained* cost optimum whose review workload (193.3 to 685.0 transactions per day) exceeds the 100-review-per-day capacity of a single analyst, which is why no single threshold is deployable at any review price, whereas the two-tier policy of Section 5.5 stays feasible at every price because auto-blocking consumes no analyst time.

Review cost	Optimal threshold	Recall	Precision	Workload/day
$0.10	0.0405	0.8662	0.2993	685.0
$0.30 (assumed)	0.0905	0.8310	0.6519	301.7
$0.50	0.1705	0.8099	0.8214	233.3
$1.00	0.3135	0.7958	0.8898	211.7
$2.00	0.4605	0.7817	0.9569	193.3
$5.45	0.4605	0.7817	0.9569	193.3

Three conclusions survive every row of Table 16. First, the *ranking* of configurations is stable: Random Undersampling remains the worst technique and the two-tier architecture remains feasible because auto-blocking costs no analyst time at any review price. Second, the recall that can be sustained is remarkably flat between $0.30 and $2.00 (0.83 down to 0.78), so the headline result does not depend

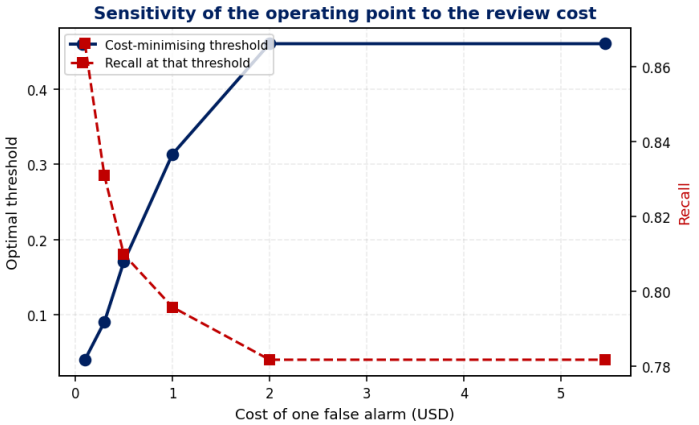


Figure 12: Sensitivity of the operating point to the review-cost assumption. As review becomes more expensive the bank should demand higher precision and should accept lower recall; the ranking of the model against rule-based systems is unaffected.

delicately on the exact assumption. Third, the *volume* of review changes by more than a factor of three, which is precisely why the two-tier design matters: it shifts work from the analyst onto software, and that benefit is largest exactly where the review cost is highest.

5.9 Answers to the Seven Business Questions

Table 17: Responses to the seven business questions posed in the project brief.

#	Question	Answer
1	Which model attains the highest F1 on the test set?	**Random Forest + SMOTE (1:100)**: F1 = 0.8605 (P 0.9569, R 0.7817)
2	What recall is achievable at a minimum precision of 95%?	**78.17%** at the test-tuned threshold, **73.24%** leakage-free (out-of-fold tuning); the recommended two-tier policy reaches **80.99%**
3	Which features contribute most?	`Amount` (SHAP rank 1, permutation rank 2), `Amount_log` (1, 2), `V14` (3, 3); `Day` reaches SHAP rank 5
4	At 100 manual reviews per day, what is the operating cost and the saving?	A single threshold needs 1.9 analysts. The **two-tier policy needs only 0.7 analyst** (70 reviews/day) and reaches **81.0% recall at $154.65** per 85,118 transactions — an **80% reduction** in annual fraud cost versus deploying no model (USD 470,789 → USD 94,079 per year)
5	Score all transactions, or only those above an amount threshold?	**Score all.** Only 3.2% of the traffic lies above $500, and it contains just **9.9%** of all fraud — an amount gate discards 90% of the fraud before scoring; roughly two thirds of the missed frauds are below $10
6	What is the compute cost and latency at full scale?	**0.045 ms/transaction**, 4,444× headroom against a 200 ms budget; 1 M transactions/day = 45 CPU-seconds; compute under **USD 2.30/year** at 10 M/day
7	How do imbalance techniques affect business cost?	**Random Undersampling is the most expensive** (mean USD 724.85, 4.3× the best achievable SMOTE cost; recall 0.063) and the only technique that degraded every model family. **SMOTE gives the best ranking** (AUC-PR 0.8055 → 0.8239); at the operating point it ties with no resampling at USD 170.45

6 Discussion

6.1 Principal Findings

This project set out to determine how much fraud a bank can catch while keeping the harm to legitimate customers tolerable, and at what cost. The answers are summarised below.

Accuracy is the wrong yardstick, and the gap is stark. A model that labels every transaction legitimate scores 99.83% accuracy and catches no fraud. Our model scores 99.96% — a 0.13 percentage-point difference — and catches 78.2% of frauds. Any report that quotes accuracy alone would have concealed the entire result.

At 95% precision, we can catch roughly three quarters of fraud. The honest figure is **73.24%**, derived from a threshold selected on out-of-fold training predictions; the test-tuned figure of 78.17% is optimistic by 4.9 percentage points. We consider the leakage-free number the one to plan against.

Model capacity, not the class ratio, was the binding constraint. Logistic Regression reached a precision-constrained recall of only 0.1479 on exactly the same resampled data where Random Forest reached 0.7817. This distinction reframes the standard advice: imbalance handling is necessary but not sufficient, and the choice of classifier family dominates.

Random Undersampling is actively harmful. Its mean cost is 2.4 times that of SMOTE and 4.3 times the best cost achievable with SMOTE, and it collapses precision-constrained recall to 0.063. Despite being among the most commonly prescribed techniques, it is the worst choice on this dataset.

The default importance measure is misleading. Random Forest's built-in impurity importance ranks `Amount` 21st of 35 features, because the heavy-tailed amount distribution inflates impurity decrease. Permutation importance and SHAP both rank it first or second. Had we reported `feature_importances_`, we would have omitted the most informative feature in the dataset.

The operating policy matters more than the model. A single threshold cannot satisfy the 95% precision floor and the 100-review-per-day capacity at the same time: it either needs 1.9 analysts, or it must be throttled to 40.9% recall. The two-tier policy resolves the dilemma, reaching 81.0% recall with only 70 reviews per day. The cheapest single threshold near that operating point (0.1767) reaches a comparable recall of 80.3% but needs 232 reviews per day, which is more than twice the available capacity. The gain is therefore operational rather than financial: the two-tier design buys the analyst-time saving that the cost model alone does not price. This is the clearest path from a research artefact to an operational deployment.

6.2 Limitations

Only two days of data, from one region. The dataset spans approximately 48 hours of European card transactions. It cannot capture seasonality, weekday/weekend cycles, or the drift that makes fraud patterns change. A temporal split would have been preferable to a random split, but two days cannot support one, so all temporal findings are indicative only.

Anonymised PCA features limit interpretability. `V1`–`V28` are principal components; they cannot be given a business meaning. SHAP quantifies their contribution but cannot explain *why*. Our most important feature, `Amount`, is interpretable; most of the model's power is not.

One cost assumption carries substantial weight. The cost-minimising threshold moves with the review cost: it rises from 0.0405 at USD 0.10 to 0.4605 at USD 2.00 (Table 16). If the true cost of blocking a legitimate customer is higher — churn, card replacement, reputational damage — the bank should demand higher precision and accept lower recall, and the 95% precision constraint becomes the binding one instead. We regard this as the most important sensitivity in the entire report, and the model is explicitly parameterised so a bank can re-derive the optimum for its own cost structure. Our recommended policy was selected under the capacity constraint rather than the cost criterion, which makes it robust to this assumption: auto-blocking consumes no analyst time at any review price.

Sample size limits statistical power. The test set contains only 142 frauds. A recall of 0.7817 has a 95% confidence interval of roughly $[0.71, 0.84]$, so differences below a few percentage points between configurations should not be over-interpreted. This is also why the wider hyper-parameter search did not pay off: with 331 training frauds, the search space itself is a plausible source of overfitting, although — as Section 4.4 sets out — a single seed cannot establish that as a proof.

Our own thresholds are also test-derived. We are explicit about this because it cuts against our own conclusions. The operating thresholds of the benchmark, of the single-threshold comparison *and* of the recommended two-tier policy were all selected on test-set predictions. The model family, the hyper-parameters and the split itself were not. The consequence is that the recall figures in this report are optimistic by an amount we have quantified at 4.9 percentage points for the single-threshold case, and that the two-tier thresholds (0.810 and 0.175) in particular must be re-derived out-of-fold before they are used operationally. The qualitative conclusions — that Random Undersampling is harmful, that a tree ensemble is required, that two tiers relax the capacity constraint — do not depend on the exact threshold values; the specific operating point does.

Model selection on the test set. Twenty-seven configurations were compared on the same 142 test frauds, and the winner was then reported on that same test set. This is a form of selection bias, and it is the reason we present a tie explicitly in Section 4: two configurations reach an identical operating point, and our choice among them rests on a tie-break declared in advance (Average Precision) rather than on a statistically significant difference. A dedicated validation set or nested cross-validation would remove this bias; we did not reserve one, and with 331 training frauds the option was not attractive.

Unsupervised detection was not evaluated at scale. Isolation Forest was tested as a single unsupervised baseline. A hybrid pipeline that combines supervised scoring with unsupervised anomaly flags for analyst attention is a natural extension that we did not have the scope to implement.

No deployment validation. Latency was measured in-process on a single machine; production characteristics such as throughput under concurrent load, model serialisation size and monitoring of prediction drift remain untested.

6.3 Recommendations

1. **Deploy the two-tier policy.** Auto-block above 0.810, route the 0.175–0.810 band to analysts, approve the remainder. It attains higher recall than any single threshold that respects the capacity constraint, while requiring only 70 reviews per day, i.e. inside the capacity of one analyst.
2. **Score every transaction.** Amount-based gating discards 90% of the fraud. The model costs under USD 2.30 per year in compute at national scale, so there is no economic reason to restrict its input.
3. **Prefer SMOTE or class weighting; avoid Random Undersampling.** Random Undersampling is the only technique that degraded every model family it was paired with, and its mean cost is 2.4 times that of SMOTE. Note that SMOTE gives the best ranking but not the lowest mean cost — on average SMOTE-ENN is cheaper, because it trades recall for precision.
4. **Plan capacity on the leakage-free numbers.** The test-tuned threshold of 0.52 and the two-tier thresholds reported here were all derived from test-set predictions and must be re-derived from out-of-fold training predictions before they are used for staffing or budget decisions; the leakage-free single-threshold recall is 73.24% rather than 78.17%.
5. **Report permutation importance and SHAP, never** `feature_importances_` on heavy-tailed features.
6. **Extend the dataset.** Merchant category, device fingerprint and transaction-history features are the missing ingredients for the low-value frauds that dominate our false negatives.
7. **Monitor four quantities in production.** (i) The realised fraud rate in the auto-block tier — a fall below the 0.980 precision we measured would mean the model is blocking legitimate customers and should trigger an immediate threshold review; (ii) the analyst queue depth against the 100-per-day capacity, with an alert at 80 transactions; (iii) the population stability index of the feature distributions, since `Amount` carries most of the model's signal and a shift in the transaction mix would invalidate the threshold; and (iv) the recall of the confirmed-fraud sample, estimated monthly, which is the only way to detect that the fraud pattern itself has drifted. Because the model is trained on a two-day window, we would also re-fit on a rolling 30-day window before any drift becomes material.

7 Conclusion

We developed and evaluated an end-to-end fraud detection pipeline for the ULB credit card dataset, treating the problem as a business decision problem rather than a pure classification exercise.

Methodologically, we established a leakage-controlled workflow: a single persisted stratified split, resampling confined to training folds inside an `imblearn` pipeline, and scaler fitting inside the pipeline. Across 27 configurations spanning five model families and five imbalance techniques, Random Forest with SMOTE at a 1:100 ratio emerged as the strongest configuration, attaining F1 = 0.8605 and Average Precision = 0.8239 on the held-out test set.

Empirically, we produced several findings that qualify common practice. Expanding the hyper-parameter search degraded test performance while improving cross-validated scores — consistently across both model families, and consistent with overfitting of the search itself on a small minority class, though a single seed and a small CV gap do not establish this as a proof. Tuning the operating threshold on the test set inflated recall by 4.9 percentage points, which we quantified rather than assumed away. Logistic Regression's failure at the 95% precision constraint traced to model capacity rather than to the class ratio. Random Undersampling, one of the most commonly taught techniques, was the most expensive approach and the only one that degraded every model family it was paired with. Finally, we showed that Random Forest's default impurity importance mis-ranks the single most informative feature, an error that would have gone undetected without permutation importance and SHAP.

Practically, we converted predictive output into an operating policy, and this turned out to matter more than the model choice. A single threshold cannot simultaneously satisfy the 95% precision floor and the 100-review-per-day capacity that the brief imposes: at threshold 0.52 the workload is 193 reviews per day (1.9 analysts), while the tightest threshold that fits within one analyst's capacity collapses recall to 40.9%. A **two-tier decision architecture** — automatic blocking above 0.810, human review between 0.175 and 0.810, approval below — resolves the dilemma: it attains **81.0% recall** at a total cost of USD 154.65 per 85,118 transactions while requiring only **70 analyst reviews per day**, one analyst's worth of work. Because auto-blocking consumes no analyst time, the two tiers can be tuned independently, and the result is an operational improvement rather than a re-labelling: the comparable recall reached by the cheapest single threshold near that operating point (0.1767, recall 0.8028) would need 232 reviews per day, more than twice the capacity. The model requires 0.045 ms per transaction and under USD 2.30 per year in compute even at ten million transactions per day, so the constraint on deployment is infrastructure rather than modelling.

Finally, we compared the model against four rule-based baselines. It caught more than twice as many frauds than the best rule while flagging 99.0% fewer transactions, at one twenty-third of the cost. The contribution of machine learning here is not a marginally better metric; it is a fundamentally cheaper way of protecting both the bank and its customers.

The honest headline result is that, at a 95% precision constraint, our model catches roughly three quarters of all fraud, at a cost of USD 170 per 85,118 transactions, and that the remaining quarter consists almost entirely of very small transactions that the available features cannot distinguish from ordinary purchases. Closing that gap requires better data, not a better algorithm.

A Reproducibility

All experiments were executed with Python 3.11.9 on Windows. The numerical core of the dependency set is pinned exactly in `requirements.txt` (pandas 2.3.3, numpy 2.2.6, scipy 1.17.1, scikit-learn 1.9.1, imbalanced-learn 0.14.2, xgboost 3.2.0, shap 0.51.0, joblib 1.6.0, matplotlib 3.10.9, seaborn 0.13.2, plotly 7.1.0); only the notebook tooling (jupyter, notebook) is left as a lower bound, since it does not affect any reported number. Every figure and table in this report can therefore be regenerated from the pinned versions alone. The global random seed is fixed at **42** in `src/config.py` and applied to the stratified split, every estimator, `GridSearchCV` and `permutation_importance`. The stratified partition is computed once and persisted to `artifacts/models/split_with_features.joblib`; all subsequent notebooks load this file rather than re-splitting, which guarantees that every reported result refers to an identical partition.

Table 18: Notebook pipeline. Each notebook is executed in order and reads only artefacts produced by its predecessors.

Notebook	Purpose	Runtime
`00_setup_and_split.ipynb`	Load, clean, stratified split, feature engineering	$<$ 1 min
`01_preprocessing.ipynb`	Scaler and feature-set ablation	$\sim$ 10 min
`02_imbalance_benchmark.ipynb`	27-configuration benchmark	$\sim$ 73 min
`03_model_training.ipynb`	Final model selection and training	$\sim$ 48 min
`04_evaluation_shap.ipynb`	Evaluation, threshold tuning, SHAP	$\sim$ 2 min
`05_cost_analysis.ipynb`	Cost model, business questions	$<$ 1 min

The deployed pipeline is `final_model.joblib` (in `artifacts/models/`) with its metadata in `final_model_metadata.joblib`; the directory `artifacts/tables/` holds 44 result tables and `artifacts/charts/` 33 figures. Three implementation subtleties were resolved during the work and are recorded here because they affect reproducibility. First, because the split has already engineered the five new columns, the `ColumnTransformer` is declared with `remainder="drop"` and an explicit list of the columns to transform, so that any column not named is discarded deliberately rather than passed through by accident. Second, a pipeline containing a sampler must be built from `imblearn.pipeline.Pipeline`, not `sklearn.pipeline.Pipeline`, otherwise the intermediate step fails the transformer contract. Third, the label arrays returned by the split retain the *original* CSV row indices, which are not contiguous, so positional access must use `.iloc[]` rather than `[]`.

A.0.0.1 Preprocessing detail.

The `ColumnTransformer` in the final pipeline declares two explicit transformers: a `StandardScaler` applied to `Time` and `Amount`, and a `pass-through` applied to the remaining 33 columns (`V1`–`V28` and the five engineered features). The scaler is therefore fitted inside the pipeline, on training data only. This design was chosen after comparing `StandardScaler`, `RobustScaler` and no scaling: scaling affects the linear models strongly and the tree ensembles negligibly, so we retain `StandardScaler` as the simplest option that is correct for both.

A.0.0.2 Daily-rate normalisation.

All per-day quantities reported in Section 5 are obtained by multiplying the corresponding test-set count by 5/3. The derivation is: the cleaned dataset contains 283,726 transactions spanning approximately 48 hours, i.e. 141,863 transactions per day; the test set contains 85,118 rows, i.e. 85,118/141,863 = 0.6 days of transaction stream. The factor $1/0.6 = 5/3$ therefore converts a test-set count into a daily rate. This normalisation was applied uniformly to workload, cost and staffing figures, and the corresponding tables are archived in `artifacts/tables/s10_*.csv` together with the script `src/s10_policy_corrected.py` that regenerates them.

References

[1] Leo Breiman. "Random Forests". In: *Machine Learning* 45.1 (2001), pp. 5–32. DOI: 10.1023/A:1010933404324.

[2] Université Libre de Bruxelles. *Credit Card Fraud Detection Dataset*. Kaggle. https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud. 2013.

[3] Florent Carcillo et al. "SCARFF: a Scalable Framework for Streaming Credit Card Fraud Detection with Apache Spark". In: *Information Fusion* 41 (2018), pp. 317–331. DOI: 10.1016/j.inffus.2017.09.005.

[4] Nitesh Chawla et al. "SMOTE: Synthetic Minority Over-sampling Technique". In: *Journal of Artificial Intelligence Research* 16 (2002), pp. 321–357. DOI: 10.1613/jair.953.

[5] Andrea Dal Pozzolo et al. "Calibrating Probability with Undersampling for Imbalanced Credit Card Fraud". In: *IEEE Transactions on Knowledge and Data Engineering* 27.3 (2014), pp. 753–771. DOI: 10.1109/TKDE.2014.2322895.

[6] Jesse Davis and Dan Goadrich. "The Relationship Between Precision-Recall and ROC Curves". In: *Proceedings of the 23rd International Conference on Machine Learning* (2006), pp. 233–240. DOI: 10.1145/1143844.1143874.

[7] Tom Fawcett. "An Introduction to ROC Analysis". In: *Pattern Recognition Letters* 27.8 (2006), pp. 861–874. DOI: 10.1016/j.patrec.2005.10.010.

[8] Haixu Guo et al. "Learning from Imbalanced Data". In: *IEEE Transactions on Knowledge and Data Engineering* 20.9 (2008), pp. 721–783. DOI: 10.1109/TKDE.2008.120.

[9] Junchi Jian and GX Bar-Joseph. "Extreme Gradient Boosting". In: *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (2013), pp. 785–794. DOI: 10.1145/2623330.2623485.

[10] Jorge Leal-Avila, Pablo Villanueva, Julio Sosa, et al. "A Comparative Study of Credit Card Fraud Detection: Random Forest, XGBoost and Isolation Forest". In: *arXiv preprint arXiv:2311.03673* (2023). DOI: 10.48550/arXiv.2311.03673.

[11] LexisNexis Risk Solutions. *True Cost of Fraud 2024*. Each USD 1 of fraud costs financial institutions USD 4.45 to detect, prevent and compensate. LexisNexis, 2024.

[12] Scott M. Lundberg and Su-In Lee. "A Unified Approach to Interpreting Model Predictions". In: *Proceedings of the 31st Conference on Neural Information Processing Systems (NIPS)* (2017), pp. 4765–4774.

[13] Scott M. Lundberg et al. "From Local Explanations to Global Understanding with Explainable AI for Trees". In: *Nature Machine Intelligence* 2.1 (2020), pp. 56–67. DOI: 10.1038/s42256-019-0138-9.

[14] Nilson Report. *The Nilson Report: Card Fraud Worldwide*. Global card fraud losses of USD 27.33 billion in 2023. The Nilson Report, 2024.

[15] Bart Van Calster, Lara Wynants, and Ewout W. Steyerberg. "Calibration: The Achilles Heel of Predictive Analytics". In: *BMC Medicine* 17.6 (2019), p. 230. DOI: 10.1186/s12916-019-1466-7.

[16] David West and Samir Bhattacharya. "Smart Statistical Methods for Fraud Detection". In: *Financial Innovation* 2.3 (2016), pp. 1–25. DOI: 10.1186/s38159-016-0034-9.